	FORMAT TUGAS LAPORAN PRAKTIKUM		
A. Mata Praktikum	Workshop Mobile Application		**Foto 3x4**
B. Dosen Pengampu	Fakhry Ikhsan Firdaus, S.Kom., M.Kom.		
C. Pokok Bahasan	Navigator, Named Routes & Passing Argument		
D. Nama Lengkap	Prasetyo Herwibowo		
E. NIM	E41250545	**NILAI**	
F. Golongan / Kelas	B		
G. Minggu / Pertemuan	Minggu 9, Pertemuan 2		

H. Kode Program
Masukkan screenshot kode program gunakan background theme putih

■ MODUL 9: INTERAKSI & PENGELOLAAN STATE DASAR

1. Lab 1: Tombol Ikuti dan Pembaruan State UI (setState)

Kode Program – Lab 1: Tombol Ikuti dan Pembaruan State UI (setState)

```
bool _isFollowing = false;

void _onFollowPressed() {
  setState(() {
    _isFollowing = !_isFollowing;
  });
}

// Render tombol secara kondisional
_isFollowing
    ? OutlinedButton.icon(
        onPressed: _onFollowPressed,
        icon: const Icon(Icons.check),
        label: const Text('Mengikuti'),
      )
    : FilledButton.icon(
        onPressed: _onFollowPressed,
        icon: const Icon(Icons.person_add),
        label: const Text('Ikuti'),
      );
```

Penjelasan Kueri/Kode: Mendeklarasikan variabel state lokal `_isFollowing` bertipe boolean di dalam class State. Handler event `_onFollowPressed` membungkus mutasi variabel dengan `setState()`, memicu pemanggilan ulang

`build()` pada framework Flutter agar UI tombol diperbarui secara reaktif.

2. Lab 2: Pembaruan Jumlah Pengikut Dinamis

Kode Program – Lab 2: Pembaruan Jumlah Pengikut Dinamis

```
int _followers = 1200;

void _setFollowing(bool value) {
  if (_isFollowing == value) return; // Mencegah mutasi redundan
  setState(() {
    _isFollowing = value;
    _followers += value ? 1 : -1;
  });
}

void _onFollowPressed() {
  _setFollowing(!_isFollowing);
}
```

Penjelasan Kueri/Kode: Mengganti teks statis jumlah pengikut dengan variabel state integer `_followers`. Mengimplementasikan method idempoten `_setFollowing(bool)` yang memvalidasi perubahan nilai sebelum menjalankan `setState()`, menjamin konsistensi mutasi data.

3. Lab 3: Feedback Pesan Snackbar & Aksi Batal (Undo)

Kode Program – Lab 3: Feedback Pesan Snackbar & Aksi Batal (Undo)

```
void _showSnack(String message, {SnackBarAction? action}) {
  ScaffoldMessenger.of(context)
    ..hideCurrentSnackBar() // Menghindari penumpukan pesan
    ..showSnackBar(
      SnackBar(
        content: Text(message),
        behavior: SnackBarBehavior.floating,
        action: action,
      ),
    );
}

// Pemanggilan handler dengan aksi Undo
_showSnack(
  'Kamu mulai mengikuti ${_profile.name}',
  action: SnackBarAction(
    label: 'BATAL',
    onPressed: () => _setFollowing(false), // Aksi pembatalan
  ),
);
```

Penjelasan Kueri/Kode: Membangun fungsi helper `_showSnack` menggunakan API resmi `ScaffoldMessenger.of(context)`. Memanggil `hideCurrentSnackBar()` sebelum menampilkan pesan baru agar tidak

terjadi penumpukan antrean, serta menambahkan `SnackBarAction` untuk fitur Undo.

4. Lab 4: Interaksi Kartu Bio Diperluas (Expandable Bio)

Kode Program – Lab 4: Interaksi Kartu Bio Diperluas (Expandable Bio)

```
bool _isBioExpanded = false;

Card(
  child: InkWell(
    borderRadius: BorderRadius.circular(12),
    onTap: () => setState(() => _isBioExpanded = !_isBioExpanded),
    child: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text('Tentang', style: textTheme.titleMedium),
          const SizedBox(height: 8),
          Text(
            _profile.bio,
            maxLines: _isBioExpanded ? null : 2,
            overflow: TextOverflow.ellipsis,
          ),
          const SizedBox(height: 8),
          Text(
            _isBioExpanded ? 'Tutup' : 'Selengkapnya',
            style: TextStyle(color: colorScheme.primary),
          ),
        ],
      ),
    ),
  ),
);
```

Penjelasan Kueri/Kode: Membungkus kartu Tentang menggunakan widget `InkWell` untuk mendeteksi event sentuhan `onTap` disertai feedback visual riak air (ripple effect). State boolean `_isBioExpanded` mengatur batasan `maxLines` pada widget `Text` secara dinamis.

5. Lab 5: Dialog Konfirmasi Modal Berhenti Mengikuti

Kode Program – Lab 5: Dialog Konfirmasi Modal Berhenti Mengikuti

```dart
Future<bool> _showConfirmDialog({
  required String title,
  required String message,
  required String confirmLabel,
}) async {
  final result = await showDialog<bool>(
    context: context,
    builder: (dialogContext) => AlertDialog(
      title: Text(title),
      content: Text(message),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(dialogContext, false),
          child: const Text('Batal'),
        ),
        FilledButton(
          onPressed: () => Navigator.pop(dialogContext, true),
          child: Text(confirmLabel),
        ),
      ],
    ),
  );
  return result ?? false; // Mengembalikan false jika ditutup di luar dial
}
```

Penjelasan Kueri/Kode: Membuat helper asinkron `_showConfirmDialog` menggunakan widget `AlertDialog` modal. Mengembalikan nilai boolean hasil seleksi pengguna melalui `Navigator.pop(dialogContext, value)` dan divalidasi dengan `if (!confirmed || !mounted) return;`.

6. Lab 6: Controlled Input Switch & Conditional Badge Chip

Kode Program – Lab 6: Controlled Input Switch & Conditional Badge Chip

```
bool _openToWork = false;

// Input Switch pada form pengaturan
SwitchListTile(
  title: const Text('Open to Work'),
  subtitle: const Text('Tampilkan lencana di kartu profilmu'),
  value: _openToWork,
  onChanged: (value) => setState(() => _openToWork = value),
);

// Tampilan kondisional badge di header profil
if (_openToWork) ...[
  const SizedBox(height: 8),
  const Center(
    child: Chip(
      avatar: Icon(Icons.work_outline, size: 18),
      label: Text('Open to Work'),
    ),
  ),
],
```

Penjelasan Kueri/Kode: Menerapkan prinsip controlled widget pada `SwitchListTile` di mana nilai komponen dikontrol penuh oleh state `_openToWork`. Pada bagian header profil, widget `Chip` ditampilkan secara kondisional menggunakan sintaks collection-if.

7. Lab 7: Checkbox Multi-Selection Keahlian & Tag Chip

Kode Program – Lab 7: Checkbox Multi-Selection Keahlian & Tag Chip

```
static const _allSkills = ['Flutter', 'Dart', 'UI/UX', 'Firebase', 'Git'
final Set<String> _skills = {'Flutter'};

for (final skill in _allSkills)
  CheckboxListTile(
    title: Text(skill),
    value: _skills.contains(skill),
    onChanged: (value) {
      setState(() {
        if (value ?? false) {
          _skills.add(skill);
        } else {
          _skills.remove(skill);
        }
      });
    },
  );
```

Penjelasan Kueri/Kode: Memanfaatkan struktur data koleksi `Set<String>` untuk mengelola item keahlian unik. Setiap

`CheckboxListTile` membaca status centang melalui `_skills.contains(skill)` dan menambahkan/menghapus item secara reaktif di dalam `setState()`.

8. Lab 8: Dropdown Enhanced Enum Status Profil

Kode Program — Lab 8: Dropdown Enhanced Enum Status Profil

```
enum ProfileStatus {
  student('Mahasiswa'),
  freelancer('Freelancer'),
  intern('Magang'),
  employee('Karyawan');

  const ProfileStatus(this.label);
  final String label;
}

DropdownButton<ProfileStatus>(
  isExpanded: true,
  value: _status,
  items: [
    for (final s in ProfileStatus.values)
      DropdownMenuItem(value: s, child: Text(s.label)),
  ],
  onChanged: (value) {
    if (value != null) setState(() => _status = value);
  },
);
```

Penjelasan Kueri/Kode: Menggunakan enhanced enum `ProfileStatus` yang memiliki atribut label untuk mencegah kesalahan penulisan string (typo). Komponen `DropdownButton` membaca daftar enum dan memperbarui teks status profil secara strongly-typed.

9. Lab 9: Refactoring Komponen _SettingsCard & Reset Dialog

Kode Program – Lab 9: Refactoring Komponen _SettingsCard & Reset Dialog

```dart
class _SettingsCard extends StatelessWidget {
  const _SettingsCard({
    required this.openToWork,
    required this.onOpenToWorkChanged,
    required this.allSkills,
    required this.selectedSkills,
    required this.onSkillChanged,
    required this.status,
    required this.onStatusChanged,
    required this.onReset,
  });
  // Data mengalir turun lewat constructor, event naik lewat callback
}

Future<void> _onResetPressed() async {
  final confirmed = await _showConfirmDialog(
    title: 'Reset pengaturan?',
    message: 'Semua pengaturan akan kembali ke nilai awal.',
    confirmLabel: 'Reset',
  );
  if (!confirmed || !mounted) return;
  setState(() {
    _openToWork = false;
    _skills..clear()..add('Flutter');
    _status = ProfileStatus.student;
  });
  _showSnack('Pengaturan berhasil direset');
}
```

Penjelasan Kueri/Kode: Melakukan modularisasi kode dengan memecah UI form pengaturan ke widget terpisah `_SettingsCard` yang stateless, menerapkan pola standar arsitektur Flutter: data mengalir turun dan event mengalir naik. Menambahkan handler reset dengan dialog verifikasi.

■ MODUL 10: NAVIGASI FLUTTER (NAVIGATOR & ARGUMENTS)

10. Lab 1: Model Data Immutability & Struktur Daftar Proyek

Kode Program – Lab 1: Model Data Immutability & Struktur Daftar Proyek

```
class Project {
  final String id, title, description;
  final List<String> techStack;
  const Project({
    required this.id,
    required this.title,
    required this.description,
    required this.techStack,
  });
}

class Profile {
  final String name, bio;
  const Profile({required this.name, required this.bio});
  Profile copyWith({String? name, String? bio}) =>
      Profile(name: name ?? this.name, bio: bio ?? this.bio);
}

// Menampilkan daftar kartu proyek pada ProfilePage
for (final project in dummyProjects)
  Card(
    child: ListTile(
      leading: const Icon(Icons.folder_open),
      title: Text(project.title),
      subtitle: Text(project.techStack.join(' • ')),
      trailing: const Icon(Icons.chevron_right),
      onTap: () => _openProjectDetail(project),
    ),
  );
```

Penjelasan Kueri/Kode: Mendefinisikan entitas domain `Project` dan `Profile` dengan sifat immutable (final field) serta method `copyWith()`. Membangun daftar kartu portofolio proyek pada `ProfilePage` menggunakan perulangan collection-for dari `dummyProjects`.

11. Lab 2: Navigasi push & pop dengan Constructor Arguments

Kode Program – Lab 2: Navigasi push & pop dengan Constructor Arguments

```dart
// Navigasi ke halaman detail dengan MaterialPageRoute
void _openProjectDetail(Project project) {
  Navigator.push(
    context,
    MaterialPageRoute(
      builder: (context) => ProjectDetailPage(project: project),
    ),
  );
}

// Pada ProjectDetailPage: Tombol pop kembali ke halaman sebelumnya
SizedBox(
  width: double.infinity,
  child: FilledButton(
    onPressed: () => Navigator.pop(context),
    child: const Text('Kembali'),
  ),
);
```

Penjelasan Kueri/Kode: Membuka halaman baru di atas tumpukan stack menggunakan `Navigator.push()` dengan transisi bawaan `MaterialPageRoute`. Data objek proyek dikirim langsung melalui constructor halaman `ProjectDetailPage`, dan ditutup memakai `Navigator.pop(context)`.

12. Lab 3: Form Edit Profil & Mengembalikan Data via pop

Kode Program – Lab 3: Form Edit Profil & Mengembalikan Data via pop

```dart
// Di EditProfilePage: Validasi & kirim data baru saat pop
void _save() {
  if (!_formKey.currentState!.validate()) return;
  final updated = widget.profile.copyWith(
    name: _nameController.text.trim(),
    bio: _bioController.text.trim(),
  );
  Navigator.pop(context, updated); // Mengembalikan objek hasil ke route as
}

// Di ProfilePage: Menunggu hasil asinkron pop dengan await
Future<void> _openEditProfile() async {
  final result = await Navigator.push<Profile>(
    context,
    MaterialPageRoute(builder: (_) => EditProfilePage(profile: _profile)),
  );
  if (!mounted || result == null) return;
  setState(() => _profile = result);
}
```

Penjelasan Kueri/Kode: Membangun form pengeditan profil yang divalidasi dengan `GlobalKey<FormState>`. Menutup

halaman form sekaligus mengirimkan objek profil baru sebagai nilai balik `Navigator.pop(context, updated)` yang ditangkap secara asinkron dengan `await`.

13. Lab 4: Pengelolaan Named Routes & Splash Screen

Kode Program — Lab 4: Pengelolaan Named Routes & Splash Screen

```
class AppRoutes {
  AppRoutes._();
  static const splash = '/';
  static const profile = '/profile';
  static const projectDetail = '/project-detail';
  static const editProfile = '/edit-profile';
}

class _SplashPageState extends State<SplashPage> {
  @override
  void initState() {
    super.initState();
    Future.delayed(const Duration(seconds: 2), () {
      if (!mounted) return;
      // Menimpa halaman splash dari stack navigasi
      Navigator.pushReplacementNamed(context, AppRoutes.profile);
    });
  }
}
```

Penjelasan Kueri/Kode: Mendefinisikan nama route dalam konstanta kelas `AppRoutes` sebagai sumber kebenaran tunggal rute aplikasi. Halaman `SplashPage` menggunakan timer 2 detik lalu menjalankan `pushReplacementNamed()` sehingga halaman splash dibuang dari stack.

14. Lab 5: Router Terpusat onGenerateRoute & Passing Arguments

Kode Program – Lab 5: Router Terpusat onGenerateRoute & Passing Arguments

```
class AppRouter {
  static Route<dynamic> onGenerateRoute(RouteSettings settings) {
    final args = settings.arguments;
    switch (settings.name) {
      case AppRoutes.splash:
        return _build(const SplashPage(), settings);
      case AppRoutes.profile:
        return _build(const ProfilePage(), settings);
      case AppRoutes.projectDetail:
        if (args is Project) return _build(ProjectDetailPage(project: args), settings
        return _build(const NotFoundPage(), settings);
      case AppRoutes.editProfile:
        if (args is Profile) return _build(EditProfilePage(profile: args), settings);
        return _build(const NotFoundPage(), settings);
      default:
        return _build(const NotFoundPage(), settings);
    }
  }
}
```

Penjelasan Kueri/Kode: Mengonsolidasikan seluruh pendaftaran rute pada `onGenerateRoute`. Pengecekan tipe argument dilakukan secara terpusat (`args is Project` dan `args is Profile`) sebelum diteruskan ke constructor halaman, mencegah terjadinya crash runtime.

15. Lab 6: Manajemen Stack Navigasi: Logout & Halaman 404

Kode Program – Lab 6: Manajemen Stack Navigasi: Logout & Halaman 404

```dart
// Logout: Membersihkan seluruh riwayat tumpukan route
void _logout() {
  Navigator.pushNamedAndRemoveUntil(
    context,
    AppRoutes.splash,
    (route) => false, // Menghapus seluruh route sebelumnya dari stack
  );
}

// Fallback jika nama route tidak ditemukan
class NotFoundPage extends StatelessWidget {
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Oops!')),
    body: Center(
      child: Column(
        children: [
          const Icon(Icons.explore_off, size: 64),
          const Text('Halaman tidak ditemukan'),
          FilledButton(
            onPressed: () => Navigator.pushNamedAndRemoveUntil(
                context, AppRoutes.profile, (r) => false),
            child: const Text('Ke Beranda'),
          ),
        ],
      ),
    ),
  );
}
```

Penjelasan Kueri/Kode: Mengatur sanitasi stack navigasi menggunakan `pushNamedAndRemoveUntil()` saat logout agar tombol back sistem tidak dapat mengakses kembali halaman privat. Menyiapkan `NotFoundPage` sebagai penanganan rute tidak dikenal (404).

16. Lab 7: Pelacakan Siklus Hidup Stack dengan NavigatorObserver

Kode Program — Lab 7: Pelacakan Siklus Hidup Stack dengan NavigatorObserver

```
class LogObserver extends NavigatorObserver {
  @override
  void didPush(Route<dynamic> route, Route<dynamic>? previousRoute) {
    debugPrint('PUSH    → ${route.settings.name} (dari ${previousRoute?.settings.name})');
  }

  @override
  void didPop(Route<dynamic> route, Route<dynamic>? previousRoute) {
    debugPrint('POP     ← ${route.settings.name} (kembali ke ${previousRoute?.settings.name})'
  }

  @override
  void didReplace({Route<dynamic>? newRoute, Route<dynamic>? oldRoute}) {
    debugPrint('REPLACE ${oldRoute?.settings.name} → ${newRoute?.settings.name}');
  }

  @override
  void didRemove(Route<dynamic> route, Route<dynamic>? previousRoute) {
    debugPrint('REMOVE  ${route.settings.name}');
  }
}
```

Penjelasan Kueri/Kode: Membangun observer navigasi kustom `LogObserver` dengan meng-override event lifecycle `didPush`, `didPop`, `didReplace`, dan `didRemove`. Memantau setiap perubahan tumpukan layar untuk kebutuhan debugging dan analitik.

17. Lab 8: Implementasi Routing Deklaratif Modern (go_router)

Kode Program – Lab 8: Implementasi Routing Deklaratif Modern (go_router)

```
final GoRouter appGoRouter = GoRouter(
  initialLocation: AppRoutes.splash,
  errorBuilder: (context, state) => const NotFoundPage(),
  routes: [
    GoRoute(
      path: AppRoutes.splash,
      builder: (context, state) => const SplashPage(),
    ),
    GoRoute(
      path: AppRoutes.profile,
      builder: (context, state) => const ProfilePage(),
      routes: [
        GoRoute(
          path: 'projects/:id', // Parameter URL dinamis
          builder: (context, state) {
            final id = state.pathParameters['id']!;
            final project = findProjectById(id);
            if (project == null) return const NotFoundPage();
            return ProjectDetailPage(project: project);
          },
        ),
      ],
    ),
  ],
);
```

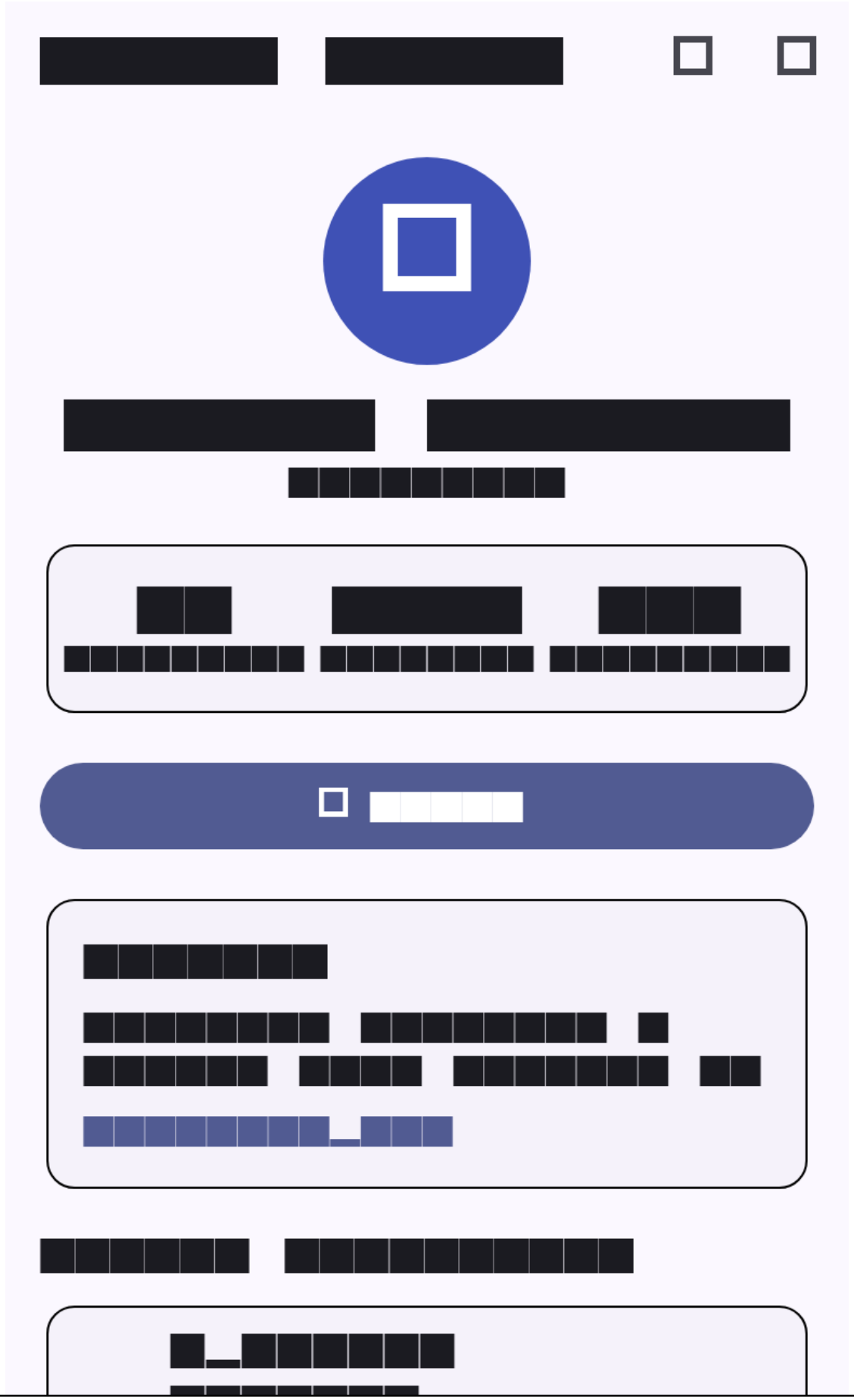
Penjelasan Kueri/Kode: Mengonversi arsitektur navigasi ke pustaka deklaratif resmi `go_router`. Mendukung struktur URL hirarkis seperti web, parameterisasi path (`:id`), deep linking, automatic stack rebuilding, dan halaman 404 terintegrasi.

https://github.com/PrasetyoHerwibowo/tugas-worksop-mobaillll	

I. Tangkapan Layar
Masukkan screenshot hasil praktikum

■ MODUL 9: INTERAKSI & PENGELOLAAN STATE DASAR — HASIL EKSEKUSI

Output — 1. Lab 1: Tombol Ikuti dan Pembaruan State UI (setState)



J. Penjelasan

Jelaskan secara singkat langkah-langkah dari tugas praktikum yang dikerjakan

Langkah 1: Lab 1: Tombol Ikuti dan Pembaruan State UI (setState)

Mendeklarasikan variabel state lokal `_isFollowing` bertipe boolean di dalam class State. Handler event `_onFollowPressed` membungkus mutasi variabel dengan `setState()`, memicu pemanggilan ulang `build()` pada framework Flutter agar UI tombol diperbarui secara reaktif. Kemudian memverifikasi bahwa tampilan antarmuka kartu profil dengan tombol aksi 'ikuti' (filledbutton) yang saat disentuh akan beralih menjadi 'mengikuti' (outlinedbutton) disertai perubahan ikon centang.

Langkah 2: Lab 2: Pembaruan Jumlah Pengikut Dinamis

Mengganti teks statis jumlah pengikut dengan variabel state integer `_followers`. Mengimplementasikan method idempoten `_setFollowing(bool)` yang memvalidasi perubahan nilai sebelum menjalankan `setState()`, menjamin konsistensi mutasi data. Kemudian memverifikasi bahwa angka statistik pengikut pada kartu profil berubah secara otomatis menjadi 1201 saat tombol ikuti ditekan, dan akan kembali menjadi 1200 saat status berhenti mengikuti.

Langkah 3: Lab 3: Feedback Pesan Snackbar & Aksi Batal (Undo)

Membangun fungsi helper `_showSnack` menggunakan API resmi `ScaffoldMessenger.of(context)`. Memanggil `hideCurrentSnackBar()` sebelum menampilkan pesan baru agar tidak terjadi penumpukan antrean, serta menambahkan `SnackBarAction` untuk fitur Undo. Kemudian memverifikasi bahwa pemberitahuan mengambang (floating snackbar) muncul di bagian bawah layar mengonfirmasi aksi ikuti, dilengkapi tombol 'batal' yang mengembalikan status dan jumlah pengikut ke kondisi semula.

Langkah 4: Lab 4: Interaksi Kartu Bio Diperluas (Expandable Bio)

Membungkus kartu Tentang menggunakan widget `InkWell` untuk mendeteksi event sentuhan `onTap` disertai feedback visual riak air (ripple effect). State boolean `_isBioExpanded` mengatur batasan `maxLines` pada widget `Text` secara dinamis. Kemudian memverifikasi bahwa teks bio yang mulanya diringkas menjadi 2 baris terpotong elipsis dan berlabel 'selengkapnya' akan memanjang menampilkan seluruh paragraf deskripsi dan berganti label menjadi 'tutup' ketika diketuk.

Langkah 5: Lab 5: Dialog Konfirmasi Modal Berhenti Mengikuti

Membuat helper asinkron `_showConfirmDialog` menggunakan widget `AlertDialog` modal. Mengembalikan nilai boolean hasil seleksi pengguna melalui `Navigator.pop(dialogContext, value)` dan divalidasi dengan `if (!confirmed || !mounted) return;`. Kemudian memverifikasi bahwa jendela dialog modal konfirmasi 'berhenti mengikuti?' muncul di tengah layar dengan latar belakang gelap, memblokir interaksi hingga pengguna memilih antara tombol 'batal' atau 'berhenti'.

Langkah 6: Lab 6: Controlled Input Switch & Conditional Badge Chip

Menerapkan prinsip controlled widget pada `SwitchListTile` di mana nilai komponen dikontrol penuh oleh state `_openToWork`. Pada bagian header profil, widget `Chip` ditampilkan secara kondisional menggunakan sintaks collection-if. Kemudian memverifikasi bahwa ketika switch 'open to work' diaktifkan pada bagian pengaturan, lencana chip bertuliskan 'open to work' lengkap dengan ikon tas kerja seketika muncul tepat di bawah status profil.

Langkah 7: Lab 7: Checkbox Multi-Selection Keahlian & Tag Chip

Memanfaatkan struktur data koleksi `Set<String>` untuk mengelola item keahlian unik. Setiap `CheckboxListTile` membaca status centang melalui `_skills.contains(skill)` dan menambahkan/menghapus item secara reaktif di dalam `setState()`. Kemudian memverifikasi bahwa kartu keahlian menampilkan tag-tag chip keahlian (flutter, dart, firebase) yang secara dinamis bertambah dan menyusun ulang posisinya menggunakan widget `wrap` saat checkbox dipilih.

Langkah 8: Lab 8: Dropdown Enhanced Enum Status Profil

Menggunakan enhanced enum `ProfileStatus` yang memiliki atribut label untuk mencegah kesalahan penulisan string (typo). Komponen `DropdownButton` membaca daftar enum dan memperbarui teks status profil secara strongly-typed. Kemudian memverifikasi bahwa menu dropdown status terbuka dan ketika opsi 'freelancer' dipilih, label status pekerjaan di bawah nama profil pada kartu langsung berubah menyesuaikan nilai baru.

Langkah 9: Lab 9: Refactoring Komponen _SettingsCard & Reset Dialog

Melakukan modularisasi kode dengan memecah UI form pengaturan ke widget terpisah `_SettingsCard` yang stateless, menerapkan pola standar arsitektur Flutter: data mengalir turun dan event mengalir naik. Menambahkan handler reset dengan dialog verifikasi. Kemudian memverifikasi bahwa dialog konfirmasi 'reset pengaturan?' muncul saat tombol reset ditekan, dan jika dikonfirmasi, seluruh pilihan switch, checkbox keahlian, dan dropdown status dikembalikan ke pengaturan pabrik.

Langkah 10: Lab 1: Model Data Immutability & Struktur Daftar Proyek

Mendefinisikan entitas domain `Project` dan `Profile` dengan sifat immutable (final field) serta method `copyWith()`. Membangun daftar kartu portofolio proyek pada `ProfilePage` menggunakan perulangan collection-for dari `dummyProjects`. Kemudian memverifikasi bahwa tampilan halaman beranda profil yang menampilkan daftar item portofolio proyek: aplikasi absensi kampus, katalog umkm jember, dan monitoring kandang pintar dengan ikon folder dan panah navigasi.

Langkah 11: Lab 2: Navigasi push & pop dengan Constructor Arguments

Membuka halaman baru di atas tumpukan stack menggunakan `Navigator.push()` dengan transisi bawaan `MaterialPageRoute`. Data objek proyek dikirim langsung melalui constructor halaman `ProjectDetailPage`, dan ditutup memakai `Navigator.pop(context)`. Kemudian memverifikasi bahwa halaman detail proyek menampilkan judul 'aplikasi absensi kampus', penjelasan lengkap proyek, daftar tag keahlian teknologi (flutter, firebase, provider), serta tombol 'kembali'.

Langkah 12: Lab 3: Form Edit Profil & Mengembalikan Data via pop

Membangun form pengeditan profil yang divalidasi dengan `GlobalKey<FormState>`. Menutup halaman form sekaligus mengirimkan objek profil baru sebagai nilai balik `Navigator.pop(context, updated)` yang ditangkap secara asinkron dengan `await`. Kemudian memverifikasi bahwa halaman edit profil yang menyediakan field input 'nama' dan 'bio' multiline dengan validasi tidak boleh kosong, tombol batal, dan tombol simpan.

Langkah 13: Lab 4: Pengelolaan Named Routes & Splash Screen

Mendefinisikan nama route dalam konstanta kelas `AppRoutes` sebagai sumber kebenaran tunggal rute aplikasi. Halaman `SplashPage` menggunakan timer 2 detik lalu menjalankan `pushReplacementNamed()` sehingga halaman splash dibuang dari stack. Kemudian memverifikasi bahwa tampilan splash screen dengan logo badge besar di bagian tengah, teks 'kartu profil digital', dan circularprogressindicator yang berjalan sebelum otomatis diarahkan ke halaman profil.

Langkah 14: Lab 5: Router Terpusat onGenerateRoute & Passing Arguments

Mengonsolidasikan seluruh pendaftaran rute pada `onGenerateRoute`. Pengecekan tipe argument dilakukan secara terpusat (`args is Project` dan `args is Profile`) sebelum diteruskan ke constructor halaman, mencegah terjadinya crash runtime. Kemudian memverifikasi bahwa halaman profil berhasil diperbarui secara reaktif dengan identitas baru (nama: prasetyo herwibowo) yang dikirim dan dikembalikan lewat named route arguments.

Langkah 15: Lab 6: Manajemen Stack Navigasi: Logout & Halaman 404

Mengatur sanitasi stack navigasi menggunakan `pushNamedAndRemoveUntil()` saat logout agar tombol back sistem tidak dapat mengakses kembali halaman privat. Menyiapkan `NotFoundPage` sebagai penanganan rute tidak dikenal (404). Kemudian memverifikasi bahwa halaman 404 not found dengan ikon explore_off dan tombol 'ke beranda' yang muncul secara elegan jika route yang dituju tidak terdaftar atau argumen tidak valid.

Langkah 16: Lab 7: Pelacakan Siklus Hidup Stack dengan NavigatorObserver

Membangun observer navigasi kustom `LogObserver` dengan meng-override event lifecycle `didPush`, `didPop`, `didReplace`, dan `didRemove`. Memantau setiap perubahan tumpukan layar untuk kebutuhan debugging dan analitik. Kemudian memverifikasi bahwa output debug console terminal merekam jejak navigasi secara kronologis: transisi push halaman, perpindahan replace splash screen, aksi pop detail proyek, hingga pembersihan stack saat logout.

Langkah 17: Lab 8: Implementasi Routing Deklaratif Modern (go_router)

Mengonversi arsitektur navigasi ke pustaka deklaratif resmi `go_router`. Mendukung struktur URL hirarkis seperti web, parameterisasi path (`:id`), deep linking, automatic stack rebuilding, dan halaman 404 terintegrasi. Kemudian memverifikasi bahwa implementasi `go_router` dengan url browser `http://localhost:5000/#/profile/projects/p1` yang

secara otomatis memetakan path parameter ':id' ke data proyek dan merekonstruksi tombol back menuju profil.

K. Kesimpulan
Beri kesimpulan dari laporan praktikum yang dikerjakan

Berdasarkan kegiatan praktikum yang telah diselesaikan, dapat disimpulkan beberapa poin penting sebagai berikut:

1. Pemahaman State dan Reaktivitas UI: State merupakan fondasi data yang menentukan tampilan antarmuka pada Flutter (UI = f(state)). Mutasi variabel state harus dibungkus di dalam pemanggilan setState() secara sinkron agar framework menjadwalkan render ulang widget tree.

2. Pola Penanganan Interaksi & Controlled Widgets: Komponen input seperti SwitchListTile, CheckboxListTile, dan DropdownButton bertindak sebagai controlled widget yang tidak menyimpan nilainya sendiri melainkan mencerminkan single source of truth dari state induk melalui prinsip 'data turun lewat constructor/value, event naik lewat callback onChanged'.

3. Umpan Balik Pengguna yang Proporsional: Penggunaan floating SnackBar dengan aksi Undo sangat ideal untuk feedback ringan yang tidak memblokir pengguna, sedangkan AlertDialog modal wajib digunakan untuk tindakan kritis dan berisiko (seperti berhenti mengikuti atau reset pengaturan) guna mencegah kesalahan aksi pengguna yang tidak disengaja.

4. Pengelolaan Tumpukan Navigasi (Navigator Stack): Konsep stack navigasi pada Flutter mengelola tumpukan layar dengan disiplin LIFO (Last In First Out). Operasi Navigator.push() menambahkan rute ke atas tumpukan, Navigator.pop() menutup rute sekaligus dapat mengembalikan data hasil, pushReplacementNamed() mengganti layar transisi seperti Splash, dan pushNamedAndRemoveUntil() mengosongkan riwayat rute saat logout untuk menjamin keamanan akses.

5. Named Routes dan Arsitektur Router Terpusat: Penerapan AppRoutes dan onGenerateRoute pada AppRouter memusatkan konfigurasi rute di satu titik, memudahkan pemeliharaan kode (maintainability), mencegah error typo rute, serta mengamankan passing arguments dengan validasi tipe data (type checking) sebelum diteruskan ke constructor halaman tujuan.

6. Komparasi Navigator Imperatif vs Deklaratif go_router: Navigator bawaan sangat tangguh untuk alur tumpukan halaman sederhana pada platform mobile, sedangkan paket deklaratif go_router menawarkan keunggulan modern berupa dukungan deep link URL berbasis web, pemetaan path parameter (:id), sinkronisasi otomatis tombol back browser, dan penanganan error 404 terpadu yang sangat esensial untuk aplikasi skala industri.